

Chapter 77

Data Engineering & Data System Design Bank

Answer the pipeline, data-modeling, and system-design questions that come up in Data Engineer interviews · design a batch or streaming pipeline live, under time pressure, the way an interviewer actually wants · reason correctly about idempotency, partitioning, and schema evolution, not just...

Approved chapter · v1

Contents

Chapter 77. Data Engineering & Data System Design Bank

77.1 Pipeline fundamentals

77.2 Basic-but-tricky data engineering questions

77.3 Data modeling for pipelines and warehouses

77.4 Orchestration and scheduling

77.5 Scale, partitioning, and performance

77.6 Data quality and monitoring

77.7 Full system design walk-throughs

Common mistakes and how to spot them

In the real world: the pipeline that failed silently for three weeks

Tools

The project

Final-week revision list

Key terms

Where this leads

Chapter 77. Data Engineering & Data System Design Bank

Part VIII — The Interview Playbook

You will learn to: answer the pipeline, data-modeling, and system-design questions that come up in Data Engineer interviews · design a batch or streaming pipeline live, under time pressure, the way an interviewer actually wants · reason correctly about idempotency, partitioning, and schema evolution, not just define the terms · walk through a full system design case end to end, stating trade-offs out loud.

How this chapter is built. Same format as Chapters 70–76: every core question leads with a "Remember it as..." hook, a one-line answer, a compact tier table. Rapid-fire sections are scan tables. **Every SQL claim in this chapter was run against live PostgreSQL 16** (the same Riverstone databases Chapter 71 uses); every DAG and dependency-resolution claim was run in real Python. Section order was planned before writing: pipeline fundamentals first, then modeling, then reliability (idempotency, failure handling), then orchestration and scale, then full system-design walk-throughs.

Learn it in pointers reference Chapters 12–13 (SQL), 16 (the star-schema/DAX material Chapter 70 already draws on), and Part V of this book (data engineering proper) at chapter level, since this chat doesn't have Part V's approved text to check exact section numbers against.

77.1 Pipeline fundamentals

Q77-001 · ETL vs. ELT: what's the actual difference, and when does it matter which you pick?

Remember it as: ETL transforms before loading, in a separate processing step. ELT loads raw data first and transforms inside the warehouse itself, using the warehouse's own compute.

Answer in one line: ETL (Extract, Transform, Load) transforms data in a separate processing layer before it ever reaches the destination system; ELT (Extract, Load, Transform) loads raw data into the destination (usually a modern cloud warehouse) first, then transforms it there using the warehouse's own compute power, which became the dominant pattern once cloud warehouses got cheap and powerful enough to make that practical.

Tier	What to say
Passes	"ETL and ELT do the same three steps in a different order" (technically true, misses why the order matters)
Strong	+ explains the practical driver: ELT keeps a full copy of raw data in the warehouse (useful for reprocessing or auditing later), and pushes transformation compute onto the warehouse rather than a separate processing cluster

Tier	What to say
Extra points	+ [Trade-offs] ELT's "load everything raw first" approach means raw, unvalidated data briefly exists in the warehouse before transformation, a real consideration for compliance-sensitive data + [Business] the choice is rarely purely technical anymore; it's often driven by which the team's existing tooling (dbt, Airflow, a specific cloud warehouse) is already built around

Likely follow-ups: Name a scenario where ETL is still clearly the better choice today. What's reverse ETL, and how does it relate to either pattern? **Red flag:** treating the two as interchangeable synonyms.
Learn it in: Part V (data engineering, this book).

Q77-002 · Batch vs. streaming: how do you decide which a new pipeline actually needs?

Remember it as: Ask how fresh the answer actually needs to be, in business terms, before assuming streaming is the more impressive, more correct choice.

Answer in one line: **Batch** processing runs on a schedule (hourly, daily), processing accumulated data in one pass, simpler to build and debug; **streaming** processes events continuously as they arrive, needed when a decision genuinely can't wait for the next batch window, and the right choice depends entirely on how fresh the downstream decision actually needs to be, not on which technology is more modern.

Tier	What to say
Passes	"Streaming is more real-time and modern, so it's generally better"
Strong	The freshness-driven decision above, with a concrete example each: a daily sales summary report is a natural batch case; a fraud-detection system blocking a transaction in real time is a natural streaming case
Extra points	+ [Business] streaming systems are meaningfully harder to build, test, and debug correctly (exactly-once semantics, out-of-order events, windowing) than batch ones; choosing streaming for a problem that doesn't actually need sub-minute freshness is real, avoidable added complexity, not a sign of engineering sophistication

Likely follow-ups: What's micro-batching, and where does it sit between the two? How would you migrate an existing batch pipeline to streaming without a risky big-bang rewrite? **Red flag:** recommending streaming by default without asking what freshness the actual decision requires.
Learn it in: Part V (this book).

Rapid-fire, 77.1

#	Question	One-line answer	Extra point
Q77-003	What's a data warehouse vs. a data lake vs. a lakehouse?	Structured, schema-on-write, optimized for analytics queries / raw, schema-on-read, stores any format cheaply / a hybrid combining a lake's flexible storage with a warehouse's structured querying and transaction support	[Learn it in] Chapter 16 (star schema) covers the warehouse modeling side directly

#	Question	One-line answer	Extra point
Q77-004	What's a data contract?	An explicit agreement between a data producer and consumer about a dataset's schema, format, and update cadence, so downstream pipelines don't silently break when the source changes	[Business] the DE-team equivalent of an API contract between two services
Q77-005	What's the difference between a full load and an incremental load?	A full load reprocesses the entire source every run; an incremental load processes only new or changed records since the last run, using a timestamp or change-tracking mechanism	[Trade-offs] full loads are simpler and self-healing but don't scale to large sources; incremental loads scale but need careful handling of updates and deletes, not just new rows
Q77-006	What's CDC (Change Data Capture)?	A technique for detecting and capturing row-level changes (inserts, updates, deletes) in a source database, often by reading its transaction log, to drive incremental pipelines without querying the whole source table	[Business] CDC avoids the "how do I know what changed" problem incremental loading otherwise has to solve with timestamps, which can miss deletes entirely

77.2 Basic-but-tricky data engineering questions

Q77-007 · Your pipeline runs on a schedule, but if it fails and gets re-run, it duplicates every row it already loaded. What's the fix?

Remember it as: A pipeline that can safely run twice with the same input and produce the same result is idempotent. One that can't, isn't safe to retry, ever.

Answer in one line: Make the load step **idempotent**: use an upsert (insert-or-update) keyed on a unique identifier instead of a plain insert, so re-running the same load with the same data updates existing rows instead of duplicating them.

Verified, live:

```
INSERT INTO daily_summary VALUES ('2025-01-05', 10000)
ON CONFLICT (order_date) DO UPDATE SET total_revenue = EXCLUDED.total_revenue;
-- run the exact same statement a second time
```

```
row count after running the same insert twice: 1  -- not 2
```

Tier	What to say
Passes	"Add a check to skip rows that already exist" (a partial fix, misses the case where a value genuinely needs updating on retry)
Strong	The upsert pattern above, explaining that idempotency means safe <i>retry</i> , not just avoiding duplicates on first run
Extra points	+ [Validate] the real proof above: the same statement run twice leaves exactly one row, not two + [Business] idempotency is what makes "just re-run the failed job" a safe, boring operational response instead of a risky one that needs careful manual cleanup first

Likely follow-ups: How would you make a multi-step pipeline (not just one insert) idempotent end to end? What's the difference between idempotency and exactly-once processing? **Red flag:** a fix that only prevents duplicates on a clean re-run, not one that's genuinely safe to retry after a partial failure mid-load. **Learn it in:** Part V (this book).

Q77-008 · What's the difference between "at-least-once," "at-most-once," and "exactly-once" delivery, and which is actually achievable?

Remember it as: *At-least-once can duplicate. At-most-once can lose data. Exactly-once, in the strict sense, usually isn't really achievable end to end, only effectively achievable by combining at-least-once delivery with idempotent processing on the receiving end.*

Answer in one line: **At-least-once** guarantees a message is delivered, possibly more than once (safe against loss, risks duplication); **at-most-once** guarantees no duplication, possibly at the cost of losing a message entirely (safe against duplication, risks loss); true **exactly-once** delivery across a genuinely distributed system is famously difficult and, in most real systems, achieved in practice by combining at-least-once delivery with idempotent processing downstream (Q77-007), not by a delivery mechanism that's magically exactly-once on its own.

Tier	What to say
Passes	Claims a specific tool or system "guarantees exactly-once" with no qualification of how
Strong	The three-way distinction above, and specifically names idempotent processing as the real mechanism behind most practical "exactly-once" claims
Extra points	+ [Depth] this is precisely why Q77-007's upsert pattern matters beyond just convenience: it's what turns an honestly at-least-once delivery guarantee into an effectively exactly-once <i>outcome</i>

Likely follow-ups: Which would you choose for a financial transaction log, and why? What's a real system that claims exactly-once, and how does it actually achieve it under the hood? **Red flag:** treating "exactly-once" as a simple guarantee some systems just have and others don't, with no understanding of how it's actually achieved. **Learn it in:** Part V (this book).

Rapid-fire, 77.2

#	Question	One-line answer	Extra point
Q77-009	What's schema drift, and why is it dangerous in an automated pipeline?	A source system's schema changing (a renamed column, a new field, a changed data type) without the downstream pipeline being updated to match, often causing a silent failure or silently wrong data rather than a loud error	[Business] a pipeline that fails loudly on schema drift is far safer than one that silently adapts (or silently drops the new column), which can hide the problem for weeks
Q77-010	What's backfilling, and why does it need special care?	Reprocessing historical data, usually after a bug fix or a new metric definition; needs care because it can be expensive at scale, and because downstream reports or dashboards using the old values need a plan for the transition, not just silently different historical numbers	[Edge cases] a backfill that changes historical numbers without any announcement is a fast way to lose a stakeholder's trust in a dashboard

#	Question	One-line answer	Extra point
Q77-011	Why can't you always trust a source system's own "last modified" timestamp for incremental loading?	Clock skew between systems, a source that doesn't update the timestamp on every field change, or a source that allows backdated edits can all make a naive timestamp-based incremental load silently miss real changes	[Learn it in] Q77-006 (CDC is the more robust alternative for exactly this reason)
Q77-012	What's a dead-letter queue?	A separate holding location for messages or records that failed processing repeatedly, so they don't block the rest of the pipeline and can be investigated separately	[Business] without one, a single malformed record can silently halt an entire stream or repeatedly retry and fail forever

77.3 Data modeling for pipelines and warehouses

Q77-013 · Implement Slowly Changing Dimension (SCD) Type 2 for a customer whose segment changes, and explain why Type 1 wouldn't preserve history

Remember it as: Type 1 overwrites, so the past is gone. Type 2 closes the old row and opens a new one, so you can always ask "what did we believe was true on this date."

Answer in one line: SCD Type 1 overwrites the old value in place, losing all history of what it used to be; SCD Type 2 keeps every historical version as its own row, with validity date ranges (and usually a current-flag), so you can query the dimension as it looked at any point in the past, not just as it looks today.

Verified, live:

```
-- Type 2: close the old row, insert a new one, instead of updating in place
UPDATE customer_history SET valid_to='2025-05-31', is_current=FALSE WHERE customer_id=1 AND is_current=TRUE;
INSERT INTO customer_history VALUES (1, 'Sharma Hardware', 'Wholesale', '2025-06-01', '9999-12-31', TRUE);
```

customer_id	customer_name	segment	valid_from	valid_to	is_current
1	Sharma Hardware	Retail	2024-01-01	2025-05-31	f
1	Sharma Hardware	Wholesale	2025-06-01	9999-12-31	t

Tier	What to say
Passes	Can define Type 1 vs Type 2 in the abstract, can't produce a working implementation
Strong	The working two-statement pattern above: close the current row, insert the new one, both inside the same transaction in practice
Extra points	+ [Validate] the real output above shows both the historical (Retail, closed) and current (Wholesale, open) rows coexisting, which is the entire point + [Business] this is exactly why "what was this customer's segment when they placed order X" is answerable with Type 2 history and not with Type 1, which would show only ever the current segment regardless of when the order was placed, silently misattributing every historical order to the wrong segment

Likely follow-ups: What's SCD Type 3, and when would you use it instead of Type 2? How would you handle a fact table that needs to join to the *correct historical version* of a Type 2 dimension, not just the current one? **Red flag:** implementing this with a plain `UPDATE` that overwrites the segment in place, silently losing history despite believing Type 2 was implemented. **Learn it in:** Chapter 16 (star schema/dimensional modeling).

Rapid-fire, 77.3

#	Question	One-line answer	Extra point
Q77-014	Star schema vs. snowflake schema?	A star schema's dimension tables are fully denormalized (flat); a snowflake schema further normalizes dimensions into sub-tables, trading some query simplicity for reduced redundancy	[Learn it in] Chapter 70, §70.9 already covers this from the BI-modeling side
Q77-015	What's a surrogate key, and why use one instead of a natural key?	A system-generated, meaningless identifier (often a sequential integer) used as a table's primary key, instead of a real-world identifier like an email or SSN that could change, be reused, or vary in format across sources	[Business] a surrogate key is exactly what makes SCD Type 2 workable: the natural business key (<code>customer_id</code>) repeats across history rows, but each row still needs its own unique surrogate key
Q77-016	What's a fact table's grain, and why must it be decided before building anything else?	The level of detail one row represents (one order line, one daily summary, one customer-month); every measure and every join in the model depends on the grain being consistent and explicit	[Edge cases] mixing grains in one fact table (some rows per-order, some pre-aggregated per-day) is a common, subtle modeling error that silently breaks any <code>SUM</code> across the whole table
Q77-017	What's a junk dimension?	A single dimension table combining several small, low-cardinality flags or indicators that don't deserve their own separate dimension tables, reducing the number of tables a fact table needs to join against	[Trade-offs] trades a slightly less "clean" model for meaningfully fewer joins in every query against the fact table

77.4 Orchestration and scheduling

Q77-018 · Given this pipeline's task dependencies, determine a valid execution order

Remember it as: A pipeline's tasks form a graph, not a list. The valid order is whatever a topological sort of that graph says it is, and there can be more than one right answer.

Answer in one line: Build a dependency graph (which tasks must finish before which others can start) and run a topological sort; any order that respects every dependency edge is valid, and there's often more than one correct answer, not a single "the" order.

Verified, live:

```
dag = {
    "extract_orders": [], "extract_customers": [],
    "clean_orders": ["extract_orders"],
    "join_orders_customers": ["clean_orders", "extract_customers"],
    "compute_daily_summary": ["join_orders_customers"],
    "load_to_warehouse": ["compute_daily_summary"],
}
# topological sort via Kahn's algorithm
```

```
valid execution order: ['extract_orders', 'extract_customers', 'clean_orders',
                        'join_orders_customers', 'compute_daily_summary', 'load_to_warehouse']
```

Tier	What to say
Passes	Describes the dependencies in English, produces an order by eyeballing it rather than a systematic method
Strong	Names topological sort explicitly and produces a genuinely valid order, acknowledging more than one valid order can exist (the two extracts could run in either order, or in parallel)
Extra points	+ [Validate] the real, algorithmically-produced order above, not a guessed one + [Business] this is exactly what a tool like Airflow does automatically from a declared DAG: the engineer states dependencies, the scheduler determines (and often parallelizes) a valid execution order, rather than the engineer hard-coding a sequence

Likely follow-ups: How would you detect a cycle in a dependency graph, meaning the DAG isn't actually valid? Which of these tasks could safely run in parallel? **Red flag:** producing an order that violates a stated dependency, or being unable to name the standard algorithm for this problem. **Learn it in:** Chapter 72A (topological concepts build on the same graph-traversal material as its BFS/DFS questions).

Rapid-fire, 77.4

#	Question	One-line answer	Extra point
Q77-019	What does DAG stand for, and why must a pipeline's dependency graph be acyclic?	Directed Acyclic Graph; a cycle (task A depends on B, which depends on A) has no valid execution order at all, since neither could ever go first	[Learn it in] Chapter 72A, Q72A-028 (BFS/DFS both need cycle-awareness for exactly this reason)
Q77-020	What's a sensor (or a "wait for" task) in an orchestration tool?	A task that waits for an external condition to become true (a file to arrive, an upstream job to finish) before the rest of the pipeline proceeds	[Business] sensors are how a pipeline safely depends on something outside its own DAG, like a third-party data delivery with no guaranteed exact timing
Q77-021	What's the risk of a pipeline with a single point of failure at its very first task?	Every single downstream task, however independent it looks, silently can't run if that one upstream task fails, making the whole day's pipeline as fragile as its weakest single link	[Business] designing for partial success (independent tasks can still run and deliver partial value even if one branch fails) is often worth the added complexity for a pipeline with many independent downstream consumers

#	Question	One-line answer	Extra point
Q77-022	What's backpressure, in a streaming context?	A mechanism for a slower downstream consumer to signal an upstream producer to slow down, preventing the consumer from being overwhelmed by data arriving faster than it can process	[Business] without backpressure handling, a slow consumer either falls further and further behind or crashes outright under load it can't keep up with

77.5 Scale, partitioning, and performance

Q77-023 · A table has grown to hundreds of millions of rows and queries filtering on date are slow. Walk through partitioning as a fix, and prove it works

Remember it as: A partitioned table lets the database skip entire partitions it knows can't contain a match, instead of scanning rows one at a time to find out.

Answer in one line: **Range partitioning** by date splits a large table into smaller physical sub-tables, each holding a specific date range; a query filtering on date lets the query planner skip (prune) any partition that can't possibly contain a match, dramatically reducing how much data actually needs to be scanned.

Verified, live:

```
CREATE TABLE events_partitioned (id SERIAL, event_date DATE, payload TEXT) PARTITION BY RANGE (event_date);
CREATE TABLE events_2025_01 PARTITION OF events_partitioned FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
CREATE TABLE events_2025_02 PARTITION OF events_partitioned FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
-- 1,000 rows loaded into each partition
EXPLAIN SELECT * FROM events_partitioned WHERE event_date = '2025-01-15';
```

```
Seq Scan on events_2025_01 events_partitioned (cost=0.00..25.00 rows=6 width=40)
  Filter: (event_date = '2025-01-15'::date)
```

The query plan touches **only the January partition**, `events_2025_01`; the February partition is never scanned at all, real, confirmed partition pruning.

Tier	What to say
Passes	"Partitioning makes queries faster" (true, no proof of the actual mechanism)
Strong	The pruning explanation above, correctly identifying that the <i>query planner</i> is what decides to skip irrelevant partitions, not the query itself doing anything different
Extra points	+ [Validate] the real query plan above shows only one partition's name (<code>events_2025_01</code>) in the plan at all, direct proof of pruning, not an assumed benefit + [Business] partitioning also makes bulk operations dramatically cheaper: dropping an entire month's stale data is <code>DROP TABLE events_2025_01</code> instead of a slow, logged <code>DELETE</code> scanning and removing millions of individual rows

Likely follow-ups: How would you choose the partition granularity (daily vs. monthly vs. yearly)? What happens to a query that doesn't filter on the partition key at all? **Red flag:** claiming partitioning always speeds up every query, without the "the query must filter on the partition key" caveat. **Learn it in:** Chapter 71, §71.10 (indexing and query optimization; partitioning is the table-level cousin of an index).

Rapid-fire, 77.5

#	Question	One-line answer	Extra point
Q77-024	What's sharding, and how is it different from partitioning?	Partitioning splits a table within one database instance; sharding splits data across <i>multiple separate database instances/machines</i> , adding a routing layer to know which shard holds which data	[Trade-offs] sharding solves a scale problem partitioning alone can't (a single machine's storage/compute limit), at the cost of real added complexity: cross-shard queries and joins become much harder
Q77-025	What's a materialized view, and when does it help at scale?	A pre-computed, physically stored query result, refreshed periodically, trading data freshness for query speed on an expensive aggregation run frequently	[Learn it in] Chapter 71, Q71-069
Q77-026	Why does <code>SELECT *</code> become a bigger problem at scale than it is on a small table?	Pulling every column, including large or unused ones, multiplies I/O cost linearly with row count; on a small table the waste is invisible, on a billion-row table it's a real, measurable cost	[Learn it in] Chapter 71, Q71-053 (the same <code>SELECT *</code> habit, here at a different scale of consequence)
Q77-027	What's a columnar storage format (like Parquet), and why does it suit analytical queries specifically?	Stores each column's values together on disk, rather than each row together; an analytical query that only needs 3 of a table's 50 columns reads only those 3 columns' data, not the other 47	[Business] this is the storage-layer reason a data warehouse or lakehouse built on columnar formats is dramatically faster for typical <code>GROUP BY</code> /aggregation-heavy analytical queries than a row-oriented format built for fast single-row lookups

77.6 Data quality and monitoring

Q77-028 · Design a data quality check that would have caught a real Riverstone data problem: a pipeline that silently produced negative revenue values

Remember it as: A data quality check needs to run automatically, on every load, and fail loudly, not depend on someone noticing a wrong number weeks later.

Answer in one line: Build automated, scheduled checks (not manual, occasional ones) that assert specific, testable properties of the data on every pipeline run, revenue should never be negative, order counts shouldn't drop to zero unexpectedly, a key column shouldn't suddenly have nulls, and fail the pipeline run loudly (alerting someone) rather than silently loading bad data.

Worked example, a concrete check:

"A simple assertion: `SELECT COUNT(*) FROM daily_summary WHERE total_revenue < 0` should always return 0; if it doesn't, fail the pipeline run and alert before the bad data ever reaches a dashboard. I'd also add a volume check: today's row count shouldn't be, say, more than 50% below the trailing 7-day average, which catches a source system silently sending a truncated file, a different kind of failure the negative-revenue check wouldn't catch at all."

Tier	What to say
Passes	"You'd check the data looks right" with no specific, automatable check named
Strong	At least one concrete, automatable assertion (the negative-revenue check above), run on every pipeline execution, not manually
Extra points	+ [Edge cases] names a <i>second</i> , different kind of check (a volume/row-count anomaly check) unprompted, since a single check type only catches one failure mode, and real pipeline failures come in more than one shape + [Business] ties the check to failing the pipeline loudly, not just logging a warning that nobody reads until the damage is already visible downstream

Likely follow-ups: How would you decide the right threshold for a volume-anomaly check without too many false alarms? What's the difference between a data quality check and a schema validation check? **Red flag:** proposing manual, human-driven spot-checks as the primary quality control for an automated pipeline. **Learn it in:** Part V (this book), and Chapter 36, §36.7's missing-value discipline (the modeling-side cousin of the same "detect before you trust" habit).

Rapid-fire, 77.6

#	Question	One-line answer	Extra point
Q77-029	What's the difference between data quality monitoring and data observability?	Data quality monitoring checks specific, predefined rules against the data itself; data observability more broadly tracks the health of the whole pipeline system (freshness, volume, schema, lineage) to catch problems you didn't think to write a specific rule for	[Business] observability is meant to catch the failure mode nobody anticipated; quality checks catch the ones you did
Q77-030	What's data lineage, and why does an on-call engineer need it at 2am?	A traceable record of where a piece of data came from and what transformed it along the way; at 2am, debugging a wrong number starts with lineage, tracing backward from the bad number to find exactly which upstream step introduced it	[Real evidence] without lineage, debugging a wrong number in a large pipeline can mean manually checking every step in order, instead of jumping straight to the likely source
Q77-031	Why alert on a pipeline's <i>absence</i> of failure alerts, not just its failures?	A monitoring system that itself silently stops running produces no alerts at all, which can be mistaken for "everything's fine" when it actually means nobody would know if something broke	[Business] a genuinely reliable monitoring setup needs a "the monitor itself is still alive" heartbeat check, not just failure-triggered alerts
Q77-032	What's the cost of alert fatigue, and how does it actually cause outages?	Too many low-value or false-positive alerts trains the on-call team to ignore or mute them, so a real, important alert gets missed in the noise exactly when it matters most	[Business] a smaller number of carefully-tuned, high-signal alerts is more reliable in practice than a comprehensive but noisy set that gets tuned out

77.7 Full system design walk-throughs

Q77-033 · Design a daily sales pipeline for Riverstone, end to end

What they're really testing: whether a complete, coherent pipeline design comes out under time pressure, touching every stage, not just the interesting parts.

Talked through live, start to finish:

"Source: the transactional order database. Extraction: a scheduled incremental pull each night, using an updated-at timestamp or CDC if available, rather than a full reload of the whole orders table every time. Landing: raw extracted data lands in a staging area first, unmodified, so I always have an unaltered copy to reprocess from if a downstream transformation bug is found later. Transformation: clean and join orders with customers and products, compute daily aggregates, handling the exact fan-out risk Chapter 71 already covers if this joins through any one-to-many relationship carelessly. Load: upsert into the daily_summary fact table, keyed on date, so a re-run is safe (Q77-007). Quality checks: the negative-revenue and volume-anomaly checks from Q77-028, run automatically before the pipeline is considered successful. Orchestration: an Airflow-style DAG with the dependency order Q77-018 worked through, scheduled to run after the source database's own nightly maintenance window closes, with a sensor task waiting for that to complete rather than a fixed clock time that could run too early. Monitoring: alert on failure, alert on the monitor itself going silent (Q77-031), and a lineage-traceable record of exactly which run produced which row in the final table."

Extra-points moves demonstrated: [Structure] covered every stage (extract, land, transform, load, quality, orchestrate, monitor) explicitly, not just the transformation logic. [Business] justified the staging-area decision with a specific reason (safe reprocessing), not just "it's best practice." [Depth] connected the design to five other specific questions already answered earlier in this same chapter, showing the concepts as one coherent system, not isolated trivia.

Likely follow-ups: How would this change if the source database couldn't support CDC at all? What's your rollback plan if a bug is found in the transformation logic three days after it shipped? **Red flag:** a design that only covers the transformation SQL, with no mention of scheduling, quality checks, or failure handling. **Learn it in:** Every section of this chapter, pulled together.

Q77-034 · Design a real-time pipeline for monitoring sensor data from Riverstone's warehouse equipment

What they're really testing: whether the candidate correctly identifies this as a genuinely streaming problem (Q77-002) and reasons about the specific challenges streaming introduces that batch doesn't have.

Talked through live, start to finish:

"First, I'd confirm this genuinely needs streaming: if the goal is alerting on equipment failure within seconds, not next-day reporting, then yes, batch wouldn't meet the actual business need here, unlike the daily sales case. Ingestion: sensors publish to a message queue (Kafka or similar), which decouples the sensors from whatever's consuming their data and buffers against a consumer temporarily falling behind. Processing: a stream processor consumes the queue, computing rolling windows (say, a 5-minute rolling average temperature per machine) rather than reacting to every single raw reading, which would be noisy. Out-of-order handling: sensor data over a real network can arrive out of order or late; I'd use event time, not processing time, for the windows, with a defined lateness tolerance, rather than assuming perfectly ordered arrival. Alerting: when a rolling average crosses a threshold, publish an alert event, itself handled idempotently (Q77-007) so a reprocessed or retried alert doesn't spam the on-call team twice for the same real event. Storage: raw sensor events also land in cheap, durable storage (a data lake) for later batch analysis and model training, alongside the real-time alerting path, since the two paths serve genuinely different purposes."

Extra-points moves demonstrated: [Clarify] explicitly validated that streaming was the right call before designing for it, rather than assuming. [Edge cases] named out-of-order and late-arriving data unprompted, a genuinely common real pitfall specific to streaming that a batch-only background can easily miss. [Business] justified the dual raw-storage-plus-real-time-path design with the different purposes each actually serves.

Likely follow-ups: How would you size the rolling window, and what trade-off does that involve? What happens if the message queue itself goes down, does data get lost? **Red flag:** a design that processes each raw sensor reading individually with no windowing, or no acknowledgment that streaming data can arrive out of order. **Learn it in:** §77.1–77.2 above, pulled together.

Q77-035 · Design a system to detect and merge duplicate customer records at scale

What they're really testing: whether "just check if the names match" gets recognized as inadequate for a real, messy production dataset, and a genuinely scalable approach is proposed instead.

Talked through live, start to finish:

"Exact-match deduplication (Chapter 71's `GROUP BY` approach) catches identical records, but real duplicates are messier: 'Sharma Hardware' vs. 'Sharma Hardware Pvt Ltd,' a trailing space, a different phone number format for the same company. I'd approach this in two stages. First, blocking: instead of comparing every pair of records against every other (which is $O(n^2)$ and doesn't scale, Chapter 72A's exact lesson), group records into smaller candidate 'blocks' likely to contain real duplicates, say by the first few characters of the normalized name plus the same city, so only records within the same block get compared pairwise. Second, within each block, score candidate pairs on similarity (normalized name distance, matching phone or email, matching address) and flag pairs above a threshold for either automatic merge (very high confidence) or human review (medium confidence). I'd deliberately avoid fully automatic merging at anything but very high confidence, since an incorrectly merged pair of genuinely different customers is a much more damaging, harder-to-undo mistake than leaving two real duplicates unmerged a little longer."

Extra-points moves demonstrated: **[Depth]** explicitly named the $O(n^2)$ scaling problem a naive pairwise-comparison approach would hit, and proposed blocking as the specific fix, directly reusing Chapter 72A's own complexity reasoning. **[Business]** made an explicit, justified risk trade-off (false-merge cost vs. missed-duplicate cost) rather than optimizing purely for automation.

Likely follow-ups: What similarity algorithm would you actually use for the name comparison? How would you handle a customer who insists two "duplicate" records are actually two genuinely different but related businesses? **Red flag:** proposing a pairwise comparison across the entire dataset with no blocking strategy, ignoring the scaling problem entirely. **Learn it in:** Chapter 71, Q71-023 (exact-match dedup) and Chapter 72A, Q72A-002 (the $O(n^2)$ cost this design specifically avoids).

Common mistakes and how to spot them

Mistake	Symptom	Fix
A load step using plain <code>INSERT</code> with no conflict handling	Re-running a failed pipeline duplicates every row	Use an idempotent upsert keyed on a unique identifier
Choosing streaming by default for a problem that doesn't need it	Real, avoidable added complexity with no business benefit	Ask how fresh the downstream decision genuinely needs to be first
SCD Type 1 used where history actually matters	Historical reports silently reflect only the current state	Use Type 2, closing old rows instead of overwriting them
A dependency graph hard-coded as one linear sequence	Independent tasks run needlessly serially, wasting time	Declare true dependencies; let the orchestrator determine (and parallelize) a valid order
Partitioning on a column queries don't actually filter on	No pruning benefit at all, despite the added complexity	Partition on the column your queries actually filter by
Manual, occasional data quality spot-checks	A bad number reaches a dashboard before anyone notices	Automated, scheduled checks that fail the pipeline loudly

Mistake	Symptom	Fix
A dedup design comparing every record pair with no blocking	Doesn't scale past a small dataset	Block first, compare only within blocks

In the real world: the pipeline that failed silently for three weeks

Arjun, a Data Engineer, is asked in an interview to describe a real production incident. He describes a pipeline that had been quietly loading zero rows for a source that had actually stopped sending data three weeks earlier, an upstream vendor had changed their export format without notice, and the pipeline's extraction step had been silently succeeding (technically, "zero rows extracted" isn't an error) and loading nothing, night after night, with no alert firing because nothing had technically failed.

He walks through exactly what he changed: a volume-anomaly check (Q77-028's second check), alerting if a day's row count dropped more than 50% below the trailing average, which would have caught this within a day instead of three weeks. He's asked the natural follow-up: why hadn't a check like that existed before the incident? His honest answer: "Because the team, including me, had only ever built checks for data that looked *wrong*, never for data that looked *absent*. It's an easy blind spot: a pipeline that runs successfully and loads nothing doesn't look like a failure from the outside, it looks exactly like a quiet day."

The interviewer's note: *"Didn't just describe the fix, explained the actual blind spot in the team's thinking that let it happen in the first place."* That's the difference this chapter's whole method is built around: a correct technical fix is table stakes; naming the reasoning gap that let the problem exist unnoticed is what shows real production experience.

Tools

PostgreSQL 16 and **Python**, the same environment Chapters 71 and 72A use; every SQL claim and every DAG/topological-sort claim in this chapter was run against them. In real production work: Airflow, Dagster, or Prefect for orchestration; Kafka or a managed equivalent for streaming; dbt for transformation; a cloud warehouse (Snowflake, BigQuery, Redshift) or lakehouse (Databricks) for storage and compute.

The project

Goal: apply this chapter's method to a real or realistic pipeline of your own.

1. Implement SCD Type 2 for one real dimension in your own data (or Riverstone's), and prove the historical version is queryable, the way Q77-013 did.

2. Make one real pipeline step idempotent using an upsert pattern, and prove a duplicate re-run doesn't duplicate rows.
3. Write two automated data quality checks for a real dataset: one checking a specific value constraint, one checking for an unexpected volume anomaly.
4. Pick one of this chapter's system design questions (daily pipeline, streaming sensors, or deduplication) and write your own full design for a different, real scenario from your own work.

Final-week revision list

Q77-001, Q77-002, Q77-007, Q77-008, Q77-013, Q77-018, Q77-023, Q77-028, Q77-033, Q77-034, Q77-035.

Key terms

ETL vs. ELT · batch vs. streaming · data warehouse vs. data lake vs. lakehouse · data contract · CDC (Change Data Capture) · idempotency · at-least-once / at-most-once / exactly-once delivery · schema drift · backfilling · dead-letter queue · SCD Type 1/2/3 · surrogate key · fact table grain · junk dimension · DAG (Directed Acyclic Graph) · topological sort · sensor (orchestration) · backpressure · partitioning · partition pruning · sharding · columnar storage · data observability · data lineage · alert fatigue · blocking (deduplication)

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.
- **Chapter 71, SQL Question Bank**, already covers the join-cardinality, indexing, and materialized-view foundations this chapter builds directly on.
- **Chapter 72A, Data Structures & Algorithms Question Bank**, supplies the complexity reasoning ($O(n^2)$ scaling, topological sort) this chapter's design questions reuse explicitly.
- **Part V of this book** (data engineering proper) teaches every technique this bank draws on, in full; this chapter tests it, it doesn't re-teach it from scratch.